

Arduino Cryptography Library

Supported algorithms

The library is split into four main sections: core, light-weight, legacy, and other.

Core algorithms

Core algorithms are found within the "libraries/Crypto" directory in the repository:

- Authenticated encryption with associated data (AEAD): [ChaChaPoly](#), [EAX](#), [GCM](#)
- Block ciphers: [AES128](#), [AES192](#), [AES256](#)
- Block cipher modes: [CTR](#), [EAX](#), [GCM](#), [XTS](#)
- Stream ciphers: [ChaCha](#)
- [Hash](#) algorithms: [SHA224](#), [SHA256](#), [SHA384](#), [SHA512](#), [SHA3_256](#), [SHA3_512](#), [BLAKE2s](#), [BLAKE2b](#) (regular and HMAC modes)
- [Hash](#) algorithm modes: [HKDF](#)
- Extendable output functions (XOF's): [SHAKE128](#), [SHAKE256](#)
- Message authenticators: [Poly1305](#), [GHASH](#), [OMAC](#)
- Public key algorithms: [Curve25519](#), [Ed25519](#), [P521](#)
- Random number generation: [RNG](#)

Reduced memory versions of some algorithms (encryption is slower, but the RAM required for the key schedule is less):

- [AESTiny128](#), [AESSmall128](#), [AESTiny256](#), [AESSmall256](#)

The "tiny" versions only support encryption which makes them suitable for the [CTR](#), [CFB](#), [OFB](#), [EAX](#), and [GCM](#) block cipher modes but not [CBC](#). The "small" versions use a little more memory but support both encryption and decryption.

Light-weight algorithms

The algorithms in the "libraries/CryptoLW" directory are new algorithms that have been designed for "light-weight" environments where memory and CPU resources are constrained:

- Authenticated encryption with associated data (AEAD): [Acorn128](#), [Ascon128](#)
- Block ciphers: [Speck](#), [SpeckSmall](#), [SpeckTiny](#)

These algorithms are fairly new, but they are ideal for Arduino devices. They don't appear in any internationally adopted standards yet but any algorithms that are adopted into standards later will be moved to the core library. Maybe you'll be the one to create that new standard!

Legacy algorithms

Legacy algorithms in the "libraries/CryptoLegacy" directory are those that should probably not be used in new protocol designs, but may be required for backwards-compatibility with older protocols:

- Block cipher modes: [CFB](#), [CBC](#), [OFB](#)
- [Hash](#) algorithms: [SHA1](#)

[CBC](#) is included in the legacy list because cryptography experts no longer recommend it for use in newer designs. It was an important mode in the past but newer designs should be using authenticated encryption with associated data (AEAD) instead. If you were looking to use [CBC](#) in your project, then please consider transitioning to one of the AEAD schemes listed above.

Over time, other algorithms may be moved from the core library to legacy.

Other algorithms

Other algorithms are provided in the remaining directories under "libraries", and consist of algorithms that are either too big for the main library, or are dedicated to a special purpose that only some applications will need:

- Post-quantum algorithms: [NewHope](#)
- Random number generation: [TransistorNoiseSource](#), [RingOscillatorNoiseSource](#)

Optimizations

All cryptographic algorithms have been optimized for 8-bit Arduino platforms like the Uno. Memory usage is also reduced, particularly for [SHA256](#) and [SHA512](#) which save 192 and 512 bytes respectively over traditional implementations. For all algorithms, static sbox tables and the like are placed into program memory to further reduce data memory usage.

[ChaCha](#) with 20 rounds and 256-bit keys is the recommended symmetric encryption algorithm because it is twice as fast as [AES128](#), constant-time, and much more secure. [AES128](#), [AES192](#), and [AES256](#) are provided for use in applications where compatibility with other systems is desirable.

If code size is an issue for your application (for example on very low end Arduino variants), then [Speck](#) on AVR is less than half the code size of [ChaCha](#), at the cost of more data memory for the state and longer key setup times. The [SpeckTiny](#) and [SpeckSmall](#) classes are even smaller at the cost of some performance when encrypting.

[BLAKE2s](#) and [BLAKE2b](#) are variations on the [ChaCha](#) stream cipher, designed for hashing, with 256-bit and 512-bit hash outputs respectively. They are intended as high performance replacements for [SHA256](#) and [SHA512](#) for when speed is critical but exact bit-compatibility of hash values is not. [BLAKE2s](#) and [BLAKE2b](#) support regular hashing, BLAKE2 keyed hashing, and HMAC modes.

Examples and other topics

- [Generating random numbers](#)
- [Using the cryptography library with ESP8266](#)

Performance

Performance on AVR

All figures are for the Arduino Uno running at 16 MHz. Figures for the Ardunino Mega 2560 running at 16 MHz are similar:

Encryption Algorithm	Encryption (per byte)	Decryption (per byte)	Key Setup	State Size (bytes)
AES128 (ECB mode)	33.28us	63.18us	158.68us	181
AES192 (ECB mode)	39.94us	76.48us	165.34us	213
AES256 (ECB mode)	46.61us	89.78us	217.79us	245
AESTiny128 (ECB mode)	40.37us		10.16us	18
AESTiny256 (ECB mode)	56.84us		17.20us	34
AESSmall128 (ECB mode)	40.37us	71.36us	134.22us	34
AESSmall256 (ECB mode)	56.84us	100.55us	177.73us	66
ChaCha (20 rounds)	14.87us	14.88us	43.74us	132
ChaCha (12 rounds)	10.38us	10.38us	43.74us	132
ChaCha (8 rounds)	8.13us	8.14us	43.74us	132
Speck (128-bit key, ECB mode)	9.74us	10.12us	253.94us	275
Speck (192-bit key, ECB mode)	10.03us	10.41us	264.63us	275
Speck (256-bit key, ECB mode)	10.31us	10.71us	275.26us	275
SpeckSmall (128-bit key, ECB mode)	33.93us	34.82us	207.66us	67
SpeckSmall (192-bit key, ECB mode)	35.20us	35.88us	220.55us	67
SpeckSmall (256-bit key, ECB mode)	36.46us	36.93us	233.32us	67
SpeckTiny (128-bit key, ECB mode)	33.93us		10.22us	35
SpeckTiny (192-bit key, ECB mode)	35.20us		13.62us	35
SpeckTiny (256-bit key, ECB mode)	36.46us		16.89us	35
AEAD Algorithm	Encryption (per byte)	Decryption (per byte)	Key Setup	State Size (bytes)
ChaChaPoly	41.20us	41.19us	902.36us	221
GCM<AES128>	109.71us	109.26us	1265.69us	284

GCM<AES192>	116.38us	115.92us	1485.56us	316
GCM<AES256>	123.04us	122.59us	1760.28us	348
GCM<Speck> (256-bit key)	86.74us	86.29us	646.88us	378
GCM<SpeckTiny> (256-bit key)	112.90us	112.44us	1225.48us	138
EAX<AES128>	71.14us	71.14us	1311.97us	268
EAX<AES256>	97.80us	97.80us	1806.57us	332
EAX<Speck> (256-bit key)	25.89us	25.88us	690.63us	362
EAX<SpeckTiny> (256-bit key)	78.20us	78.20us	1269.19us	122
Acorn128	20.39us	20.06us	4817.82us	60
Ascon128	42.71us	43.07us	738.68us	60
Hash Algorithm	Hashing (per byte)	Finalization		State Size (bytes)
SHA1	21.86us	1421.86us		95
SHA256	43.85us	2841.04us		107
SHA512	122.82us	15953.42us		211
SHA3_256	60.69us	8180.24us		205
SHA3_512	113.88us	8196.34us		205
BLAKE2s	20.65us	1335.25us		107
BLAKE2b	65.22us	8375.34us		211
Authentication Algorithm	Hashing (per byte)	Finalization	Key Setup	State Size (bytes)
SHA1 (HMAC mode)	21.86us	4290.62us	1418.49us	95
SHA256 (HMAC mode)	43.85us	8552.61us	2836.49us	107
BLAKE2s (Keyed mode)	20.65us	1335.25us	1339.51us	107
BLAKE2s (HMAC mode)	20.65us	4055.56us	1350.00us	107
BLAKE2b (Keyed mode)	65.22us	8375.34us	8357.25us	211
Poly1305	26.26us	489.11us	17.06us	53
GHASH	74.59us	15.91us	14.79us	33
XOF Algorithm	Hashing (per byte)	Extending (per byte)	Encryption (per byte)	State Size (bytes)
SHAKE128	49.43us	49.02us	49.59us	206
SHAKE256	60.77us	60.37us	60.93us	206
Public Key Operation	Time (per operation)	Comment		

Curve25519::eval()	2716ms	Raw curve evaluation
Curve25519::dh1()	2718ms	First half of Diffie-Hellman key agreement
Curve25519::dh2()	2717ms	Second half of Diffie-Hellman key agreement
Ed25519::sign()	5148ms	Digital signature generation
Ed25519::verify()	8196ms	Digital signature verification
Ed25519::derivePublicKey()	5102ms	Derive a public key from a private key
P521::eval()	46290ms	Raw curve evaluation
P521::dh1()	46293ms	First half of Diffie-Hellman key agreement
P521::dh2()	46304ms	Second half of Diffie-Hellman key agreement
P521::sign()	60514ms	Digital signature generation
P521::verify()	109078ms	Digital signature verification
P521::derivePublicKey()	46290ms	Derive a public key from a private key
NewHope::keygen() , Ref	639ms	Generate key pair for Alice, Ref version
NewHope::sharedb() , Ref	1237ms	Generate shared secret and public key for Bob, Ref version
NewHope::shareda() , Ref	496ms	Generate shared secret for Alice, Ref version
NewHope::keygen() , Torref	777ms	Generate key pair for Alice, Torref version
NewHope::sharedb() , Torref	1376ms	Generate shared secret and public key for Bob, Torref version
NewHope::shareda() , Torref	496ms	Generate shared secret for Alice, Torref version

Where a cipher supports more than one key size (such as **ChaCha**), the values are typically almost identical for 128-bit and 256-bit keys so only the maximum is shown above.

Due to the memory requirements, **P521** and **NewHope** performance was measured on an Arduino Mega 2560 running at 16 MHz. They are too big to fit in the RAM size of the Uno.

Performance on ARM

All figures are for the Arduino Due running at 84 MHz:

Encryption Algorithm	Encryption (per byte)	Decryption (per byte)	Key Setup	State Size (bytes)
AES128 (ECB mode)	6.58us	11.40us	38.15us	188
AES192 (ECB mode)	7.94us	13.83us	39.79us	220
AES256 (ECB mode)	9.30us	16.25us	49.68us	252
AESTiny128 (ECB mode)	7.23us		1.25us	20
AESTiny256 (ECB mode)	10.62us		1.43us	36

AESSmall128 (ECB mode)	7.23us	12.33us	23.44us	36
AESSmall256 (ECB mode)	10.62us	16.92us	31.88us	68
ChaCha (20 rounds)	0.87us	0.88us	4.96us	136
ChaCha (12 rounds)	0.70us	0.71us	4.96us	136
ChaCha (8 rounds)	0.62us	0.62us	4.96us	136
Speck (128-bit key, ECB mode)	0.97us	0.96us	36.80us	288
Speck (192-bit key, ECB mode)	1.00us	0.98us	38.14us	288
Speck (256-bit key, ECB mode)	1.03us	1.01us	39.31us	288
SpeckSmall (128-bit key, ECB mode)	2.72us	2.30us	26.89us	80
SpeckSmall (192-bit key, ECB mode)	2.80us	2.39us	27.80us	80
SpeckSmall (256-bit key, ECB mode)	2.90us	2.48us	29.08us	80
SpeckTiny (128-bit key, ECB mode)	2.72us		1.47us	48
SpeckTiny (192-bit key, ECB mode)	2.81us		1.54us	48
SpeckTiny (256-bit key, ECB mode)	2.90us		1.83us	48
AEAD Algorithm	Encryption (per byte)	Decryption (per byte)	Key Setup	State Size (bytes)
ChaChaPoly	1.71us	1.71us	45.08us	240
GCM<AES128>	10.90us	10.90us	248.83us	312
GCM<AES192>	12.30us	12.31us	296.83us	344
GCM<AES256>	13.66us	13.67us	350.25us	376
GCM<Speck> (256-bit key)	5.27us	5.28us	75.31us	408
GCM<SpeckTiny> (256-bit key)	7.06us	7.07us	94.20us	168
EAX<AES128>	12.33us	12.33us	234.91us	280
EAX<AES256>	16.99us	16.99us	322.92us	344
EAX<Speck> (256-bit key)	2.80us	2.80us	81.63us	384
EAX<SpeckTiny> (256-bit key)	6.69us	6.69us	110.91us	144
Acorn128	0.75us	0.75us	175.70us	64
Ascon128	3.52us	3.50us	51.67us	72

Hash Algorithm	Hashing (per byte)	Finalization		State Size (bytes)
SHA1	0.94us	62.34us		112
SHA256	1.15us	76.60us		120
SHA512	2.87us	370.37us		224
SHA3_256	5.64us	735.29us		224
SHA3_512	10.42us	735.49us		224
BLAKE2s	0.80us	53.39us		120
BLAKE2b	1.28us	164.66us		224
Authentication Algorithm	Hashing (per byte)	Finalization	Key Setup	State Size (bytes)
SHA1 (HMAC mode)	0.94us	196.74us	68.06us	112
SHA256 (HMAC mode)	1.15us	238.98us	80.44us	120
BLAKE2s (Keyed mode)	0.80us	53.39us	55.10us	120
BLAKE2s (HMAC mode)	0.80us	168.20us	57.60us	120
BLAKE2b (Keyed mode)	1.28us	164.66us	166.68us	224
Poly1305	0.81us	19.01us	2.57us	60
GHASH	4.47us	1.52us	2.60us	36
XOF Algorithm	Hashing (per byte)	Extending (per byte)	Encryption (per byte)	State Size (bytes)
SHAKE128	4.60us	4.45us	4.59us	232
SHAKE256	5.64us	5.49us	5.63us	232
Public Key Operation	Time (per operation)	Comment		
Curve25519::eval()	103ms	Raw curve evaluation		
Curve25519::dh1()	103ms	First half of Diffie-Hellman key agreement		
Curve25519::dh2()	104ms	Second half of Diffie-Hellman key agreement		
Ed25519::sign()	195ms	Digital signature generation		
Ed25519::verify()	306ms	Digital signature verification		
Ed25519::derivePublicKey()	194ms	Derive a public key from a private key		
P521::eval()	1503ms	Raw curve evaluation		
P521::dh1()	1503ms	First half of Diffie-Hellman key agreement		
P521::dh2()	1503ms	Second half of Diffie-Hellman key agreement		
P521::sign()	1860ms	Digital signature generation		

P521::verify()	3423ms	Digital signature verification
P521::derivePublicKey()	1503ms	Derive a public key from a private key
NewHope::keygen() , Ref	29ms	Generate key pair for Alice, Ref version
NewHope::sharedb() , Ref	41ms	Generate shared secret and public key for Bob, Ref version
NewHope::shareda() , Ref	9ms	Generate shared secret for Alice, Ref version
NewHope::keygen() , Torref	42ms	Generate key pair for Alice, Torref version
NewHope::sharedb() , Torref	53ms	Generate shared secret and public key for Bob, Torref version
NewHope::shareda() , Torref	9ms	Generate shared secret for Alice, Torref version